Palm vein module SDK user documentation

- Palm vein module SDK user documentation
  - 1.1 Copy the aar file to the libs folder and put the .bin resource file into the assets directory;
  - 1.2 Add to dependencies in build.gradle
  - 2. SDK Authorization
  - 2.1 Authorization through modules
  - 2.2 Authorization via the Internet
  - 2.3 Authorization by secure chip
- 3. Find USB devices
  - 3.1 Initialize USB Device Manager
  - 3.2 USB monitoring callback
- 4. Initialize the device
  - 4.1 Initialize in root mode
  - 4.2 Non- root initialization (recommended)
- 5. Initialize SDK
- 6. Palm vein registration
  - 6.1 Start recording mode
  - 6.2 Setting up image capture and registering callbacks
  - 6.3 End of registration
  - 6.4 Register via Image
    - 6.4.1 Single -mode module (910,915,920)
    - 6.4.2 Multimodal module (960,960s)
- 7. Add or delete palm veins
  - 7.1 Initialize the base library
  - 7.2 Adding features to the base library
  - 7.3 Deleting features from the base database
  - 7.4 Clear the features in the base database
  - 7.5 Query the number of features in the database
- 8. Palm vein recognition
  - 8.1 Setting up image capture and registering callbacks
  - 8.2 Base database comparison (1:N)
  - 8.3 Feature Comparison (1:1)
- 9. Parameter settings
- 10. Algorithm version number
  - 10.1 Get the algorithm version number
  - 10.2 Determine whether the algorithm version is compatible
- 10. Non-module usage
- 12. Instructions for use of 960 module
- 13. Error Code

- 13.1 Initialization and parameter transfer errors
- 13.2 Algorithm Related
- 13.3 Authorization Error

XIV. Appendix

- 14.1 Identification Process
- 14.2 Entry Process

—, Import SDK

1.1 Copy the aar file to the libs folder, put the .bin resource file into the assets directory;

1.2 Add to build.gradle 's dependencies

```
implementation(fileTree("libs"));
```

II、 SDK authorization

PV920 can be authorized by module PV910 , PV720 via network authorization

2.1 Authorization through modules

No additional action required, but you must initialize the device first and then initialize the SDK, and needs to be done through JXP almApi.ANDROID_915_OTG_DEVICE Initialize the device, 920 and 960 series recommend this authorization method

```
//915,920
int retVal = JXPalmTool.getPal
    mInstance().initializeDevice( this, device,
    JXPalmApi.ANDROID_ 915_OTG_DEVICE);

//960,960s
int retVal = JXMultimodalPalmTool.getP
    almInstance().initializeDevice( this, device,
    JXMultimodalPalmApi.ANDROI D_960_OTG_DEVICE);
```

```
int ret = JXPalmTool.getPalmI nstance().initializeSDK(...);
```

2.2 Authorization via the Internet

You need to obtain authorization before use , including APP_ID and SIGN_KEY . Networking permission and storage read and write permission are required when authorizing the device. Add the following statement to AndroidMainifest.xml :

```
<uses-permission android:name="android.permission.ACCES S_NETWORK_STATE" />
<uses-permission android:name="android.permission.READ_E XTERNAL_STORAGE"
                 />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

授权代码如下

```
int result = JXLicense.init(getApplicationContext());
if(result == 0){
    initPalmSDK();
}else{
    JXAuthen.getKey(APP_ID, SIGN_KEY, new JXAuthen.AuthenListener() {
        @Override
        public void complete(int code, String message, String data) {
            int result = JXLicense.s etAuthFile(getApplicationContext(),
            data); initPalmSDK();
        }
    });
}
```

2.3 Authorization through secure chip

```
byte[] code  = new byte[112];
byte[] license  = new byte[224];
JXLicense.getICAuthenCode(code, license);
String codeStr = Encryption chip decryption
String modelStr = Encryption chip decryption
int ret = JXLicense .setICAuthenLicense(codeStr, modelStr);
```

3. Find USB devices

3.1 Initialize USB Device Manager

```
JXPalmUSBManager usbManager =
    new JXPalmUSBManager(this.getApplicationContext (),usbListener);
```

```
usbManager.registerUSBPermissionReceiver();
usbManager.initUSBPermission();
```

3.2 USB monitoring callback

When the palm vein device is found, it will be called back in onCheckPerm ission, the device will be called back in onUSBRemoved after being disconnected, after the device is inserted onUSBArrived callback

```
private JXPalmUSBManagerListener usbListener =
    new JXPalmUSBManagerListener(){
        public void onCheckPermission (int result, UsbDevice
        device){} public void onUSBArrived(UsbDevice device) {}
        public void onUSBRemoved(UsbDevice device) {}
    }
```

4. Initialize the device

4.1 Initialize in root mode

Get usb permission Different Android motherboards may have different ways of obtaining permissions ., the following is for reference only:

```
static public void getPermission() {
    String[] commands = new String[] {"chmod -R 777 /dev/bus/usb" };
    Process process = null;
    DataOutputStream dataOutputStream = null;
    try {
        process = Runtime.getRuntime().exec("su");
        dataOutputStream = new DataOutputStream(process.getOutputStream());

        int length = commands.length;
        for (int i = 0; i < length; i++) {
            dataOutputStream.writeBytes(commands[i] + "\n");
        }
        dataOutputStream.writeBytes("exit\n");
        dataOutputStream.flush();
        process.waitFor();
        Log.i("USBPermissionManager", "getPermission success");
    } catch (Exception e) {
        FileLogUtil.e("USBPermissionManager", "Exception " + e);
    } finally {
        try {
            if (dataOutputStream != null) {
                dataOutputStream.close();
            }
            process.destroy();
        } catch (Exception e) {
```

```
        }
    }
}
```

Initialize the palm vein device, you need to obtain USB permissions to successfully open the device:

```
int retVal = JXPalmTool.getPal
    mInstance().initializeDevice( this, device, NULL);
```

4.2 Non- root initialization (recommended)

The last parameter cannot be empty, the options are ANDROID_910 _DEVICE, ANDROID_915_OTG_DEVICE , ANDROID_960_OTG_DEV ICE

```
int retVal = JXPalmTool.getPal mInstance().initializeDevice(
    this, device, JXPalmApi.ANDROID_ 915_OTG_DEVICE);
```

5. Initialize SDK

Initialization needs to be performed after authorization is completed, the initialization process is a time-consuming operation, it is recommended to put it in an asynchronous thread for initialization, the path needs to refer to the actual International file name

```
int ret = JXPalmTool.getPalmInstance
        ().initializeSDK(this,
        "palm/palm_detection_0904_en.bin",
        "palm/palm_filter_en.bin ",
        "palm/palm_landmark_0904_en.bin",
        "palm/palm_ir_liveness_20231215_en.bin",
        "palm/palm_vein_feature_1121_2_en.bin");
```

6. Palm vein registration

6.1 Start recording mode

```
JXPalmTool.getPalmInstance().startEnroll();
```

6.2 Setting up image capture and registering callbacks

Set the callback function:

```
JXPalmTool.getPalmInstance( ).setEnrollListener(listener);
```

Start capturing images . If you want to adjust the distance detection rate or image capture rate, you need to add blocking code in the callback function,For example Thread.Sleep()

```
private JXPalmCaptureListener mCaptureListener = new JXPalmCaptureListener() {
    @Override
    public void onDistance(int i) {
    }

    @Override
    public void onCapture(int code, JX imax jxImage) {
    }
};

JXPalmTool.getPalmInstance().s tartCapture(mCaptureListener);
```

Parameter name	Parameter Type	More Information
code	int	Error code , 0 success , other failures
image	JXImage	Width 480 , high 640

grayscale image Callback function:

```
final JXPalmEnrollListener listener = new JXPalmEnrollListener() {
    @Override
    public void onEnroll(int code, int index, byte[] feature, float
    motion, JXImage image) {}

    @Override
    public void onDetected(int code, int[] rect, JXImage
    jxImage) {}

    @Override
    public void onLiveness(int code , float score)
{} };
JXPalmTool.getPalmInstance().startEnroll();
```

onEnroll

Parameter name	Parameter Class type	More Information

code	int	Error Code, 0 success, 359 Incomplete palm, 354 , 358 The palms are too far apart, 352 The palm brightness is too dark, 353 Palm brightness is too bright, 355 The palm shape does not meet expectations (such as tilted, closed, etc.)

Parameter name	Parameter Type	More Information
index	int	Input subscript , indicating the number of inputs
feature	int	The length of the vein feature is 2048 bytes
image	JXImage	Width 480 , high 640 grayscale image

onDetected

Parameter name	Parameter Type	More Information
code	int	Error Code , 0 success, other failures
rect	int[]	Palm position [x1, y1, x2, y2, x3, y3, x4, y4]
image	JXImage	Width 480 , high 640 grayscale image

onLiveness

Parameter name	Parameter Type	More Information
code	int	Error code , 0 success, other failures
score	float	Live fraction , greater than 0.8 is alive

6.3 End of registration

```
JXPalmTool.getPalmInstance().cancelEnroll();
```

6.4 Register via Image

When the registered image comes from a server or other acquisition device , you can register via picture

6.4.1 Single-mode modules (910,915,920)

```
JXPalmFeature palmFeature = XPalmTool.getPalmIn stance().getFeature(Bitmap
bitmap, int level);
JXPalmTool.getPalmInstance().ad dPalm(String group, String
    uid, int index, palmFeature .feature);
```

6.4.2 Multimodal module (960,960s)

```
JXPalmFeature palmFeature =
JXMultimodalPalmTool.getPalmInstance().getFeat ure(Bitmap bitmap, int level);
```

```
JXMultimodalPalmTool.getPalmInstance().addP alm(String group, String uid,
int index, palmFeature.feature);
```

7. Add or delete palm veins

7.1 Initialize the base library

When you need to split the library, you can initialize multiple libraries

```
JXPalmTool.getPalmInstanc e().initGroup(String group);
```

Parameter name	Parameter Type	More Information
group	String	Compare the group names of the base database

7.2 Add features to the base library

Note that this method does not store data permanently., stored only in memory, for 1:N comparison, persistent storage needs to be implemented by the application.

```
JXPalmTool.getPalmInstance().ad dPalm(String group, String
    uid, int index, byte[] feature);
```

Parameter name	Parameter Type	More Information
group	String	Compare the group names of the base database
uid	String	Palm vein ID or user ID
index	int	Palm type, the left hand is 0, the right hand is 1, Not strictly limited
feature	byte[]	Palm vein characteristics

Note: a. Uid and index form a key, when the key is repeated, will overwrite features with the same key value. b. Please *do not* include the string "_@_" in uid . Otherwise it will affect the userId mapping

7.3 Deleting Features from the Base Database

group, uid, index must strictly correspond to addPalm

```
JXPalmTool.getPalmInstance(). delPalm(String group, String
    uid, int index);
JXPalmTool.getPalmInstance().delPalmByU serId(String group, String uid);
```

7.4 Clear the features in the base database

```
JXPalmTool.getPalmInstance ().clearPalms(String group);
```

7.5 Query the number of features in the database

```
JXPalmTool.getPalmInstanc e().palmCount(String group);
```

Eight, Palm vein recognition

8.1 Setting up image capture and registering callbacks

Set the callback function:

```
JXPalmTool.getPalmInstance( ).setMatchListener(listener);
```

Start capturing images

```
JXPalmTool.getPalmInstance() .startCapture((code, image)-
    >{ if (0 == code) {
        Bitmap bitmapPalm = image.toBitmap
    (); // Display image
   }
});
```

Note: jxImage is not used or has been used up., please call jxImage.recycle(), to improve memory usage efficiency, to avoid frequent triggering of GC .

回调函数:

```
final JXPalmMatchListener mPalmMatchListener = new JXPalmMatchListener() {
        @Override
        public void onMatch(int code, byte[] feature, JXImage jxImage) {}

        @Override
        public void onDetected(int code, int[] rect, JXImage jxImage) {}

        @Override
        public void onLiveness(int result, float score) {}
    };
```

onMatch

Parameter name	Parameter Type	More Information
code	int	Error Code, 0 success, other failures
feature	int	The length of the vein feature is 2048 bytes
image	JXImage	Width 480, high 640 grayscale image

Note: jxImage is not used or has been used up., please call jxImage.recycle(), to improve memory usage efficiency, to avoid frequent triggering of GC .

onDetected

Parameter name	Parameter Type	More Information
code	int	Error Code, 0 success, other failures
rect	int[]	Palm position [x1, y1, x2, y2, x3, y3, x4, y4]
image	JXImage	Width 480, high 640 grayscale image

Note: jxImage is not used or has been used up., please call jxImage.recycle(), to improve memory usage efficiency, to avoid frequent triggering of GC .

onLiveness onLiveness

Parameter name	Parameter Type	More Information
code	int	Error code , 0 success, other failures
score	float	Live fraction , greater than 0.8 is alive

8.2 Base database comparison (1:N)

```
int retVal = JXPalmTool.getPa                                    group, byte[] feature,
lmInstance().palmMatch(String String[] uuids);
```

Parameter Name	Parameter Type	Details
group	String	Compare the group names of the base database
feature	byte[]	Palm vein characteristics
uuids	String[]	Palm vein ID or user ID , Current version returns top1 , where uuids[0] is id and uuids[1] is index
return int		greater than 0 is the comparison score , otherwise it is an error code , The threshold is

usually set between 60 and 70, greater than the threshold is considered passed

8.3 Feature Comparison (1:1)

```
int retVal = verify(byte[] regFeature, byte[] ver Feature);
```

Parameter name	Parameter Type	More Information
regFeature	byte[]	Registration Features
verFeature	byte[]	Identification features
return	int	A value greater than 0 is a comparison score., otherwise it is an error code

9. Parameter settings

JXPalmConfig

Parameter Name	Parameter Type	Details
IrLivenessEnable	boolean	IR liveness enabled?
Rotation	int	Rotation Angle
HorizonFlip	boolean	Horizontal mirror
VerticalFlip	boolean	Mirror vertically
MinEnrollQuality	int	The minimum recording quality allowed
MinEnrollBrightness	int	The minimum recording brightness allowed
MaxEnrollBrightness	int	Maximum recording brightness allowed
MaxEnrollRoll	int	Maximum rotation angle allowed for recording
MaxEnrollYaw	int	Maximum horizontal rotation angle allowed for recording
MaxEnrollPitch	int	Maximum horizontal rotation angle allowed for recording
MinEnrollSize	int	Minimum size allowed for recording
MinEnrollOccupyRatio	float	Minimum percentage allowed for recording
MinPointResponse	float	The minimum key point response allowed to be entered
MinVerifyQuality	int	Minimum image quality
MinVerifyBrightness	int	Minimum brightness
MaxVerifyBrightness	int	Maximum brightness

MaxVerifyRoll	int	Maximum rotation angle
MaxVerifyYaw	int	Maximum horizontal rotation angle
MaxVerifyPitch	int	Maximum vertical rotation angle
MinVerifySize	int	Minimum verification size
MinVerifyOccupyRatio	float	Minimum verification ratio
LivenessThreshold	float	Liveness threshold

Ten, Algorithm version number

10.1 Get the algorithm version number

```
JXPalmApi.getAlgoVersion()
```

Return value example: 2.0.0 major.minor.patch

10.2 Determine whether the algorithm version is compatible

```
JXPalmApi.isAlgoVersionCompatible("2.1.0")
```

If it returns true, it indicates compatibility, otherwise it is not compatible In principle, the major version remains unchanged, the extracted features are compatible.

Ten— Non-module usage

Non-modules do not need to initialize the device, that is, initializeDevice is not called, but to start the worker thread

```
JXPalmTool.getPalmInstance().startWork()
```

If you need to stop the working thread , call the following function

```
JXPalmTool.getPalmInstance().stopWork()
```

After starting the worker thread , images need to be input from outside

```
JXPalmTool.getPalmInstance().add DetectImage(JXImage image);
```

JXImage can be converted from

```
JXPalmTool.getPalmInstance().yuvToJXImage(byte[ ] data, int width, int height);
```

Twelve、 960 module usage instructions

replace JXPalmTool in the above section with JXMultimodalPalmTool, no other changes are required

13. Error Codes

13.1 Initialization and parameter transfer errors

Error Code	More Information
101	Parameter length error
102	Wrong resource path
103	SDK not initialized
104	Parameter error
105	Parameter not supported
106	Initialization Error
107	Handle Error
109	Memory Error
201	The number of image channels is not supported.

13.2 Algorithm Related

Error Code	More Information
301	Palm not detected
302	Palm filter failed
303	Live body not passed
351	Image quality failed
352	Image brightness is too low
353	Image brightness is too high
354	The palm is too small
355	Palm gesture detection failed
358	The palm area is too small

359	Incomplete palm

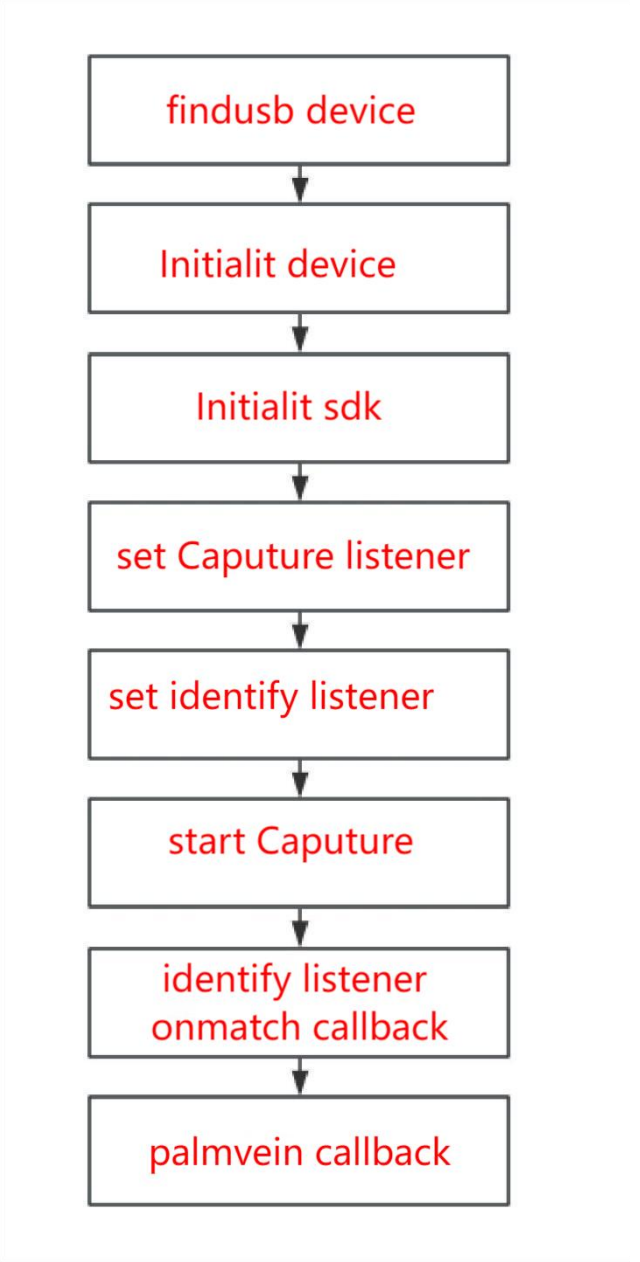
13.3 Authorization Error

Error Code	More Information
10001	Invalid authorization code
10002	Authorization code expired

Error Code	More Information
10003	Authorization code error
10004	Authorization code fingerprint matching failed
1 0006	Network authorization failed
10009	Failed to authorize by device

14.appendix

14.1 Identification process



14.2 Entry Process

